SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Vasantha Nithi D Y	**Reg. No.:**	192424137
Section / Batch:	2024	**Date:**	

Code of Conduct

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.

4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.
Apply your algorithm to:
User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.

5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	3	4	4	4	19	93
2	4	4	3	4	4	19	
3	3	4	4	3	4	18	
4	4	3	4	4	3	18	
5	3	4	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 — Reference Resolution Using Multiple Constraints

Given:

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

Pseudocode

```
ALGORITHM ReferenceResolution(text)

INPUT: Discourse text
OUTPUT: Resolved discourse

1. Extract all entities and referring expressions.

2. entities ← {Meena, Kavya, laptop, project}

3. For each referring expression R:
    candidates ← FindPossibleAntecedents(R, entities)

    For each candidate C:
      score(C) ← 0

      IF gender(R) = gender(C)
        score(C) ← score(C) + 1

      IF number(R) = number(C)
        score(C) ← score(C) + 1

      IF C is recently mentioned
        score(C) ← score(C) + 1

      IF grammaticalRole(C) is compatible
        score(C) ← score(C) + 1
```

IF semanticCompatibility(R, C) = TRUE

score(C) ← score(C) + 1

R.antecedent ← candidate with highest score

4. Replace each referring expression with its selected antecedent.

5. Return resolved discourse.

END ALGORITHM

Apply to the Given Discourse

Expression	**Possible antecedents**	**Selected**
she	Meena, Kavya	**Kavya**
one	laptop	**laptop**
her	Meena, Kavya	**Kavya**
it	laptop	**laptop**

The intended interpretation is:

Meena gave Kavya a laptop

↓

Kavya needed the laptop for her project

↓

Kavya thanked Meena

↓

Kavya started using the laptop

Resolved Discourse

Meena gave Kavya a laptop because Kavya needed the laptop for Kavya's project. Kavya thanked Meena and started using the laptop.

How the Algorithm Avoids Incorrect Resolution

When multiple candidates have the same gender and number, the algorithm does **not rely only on agreement**. It combines:

Gender

+

Number

+

Recency

+

Grammatical Role

+

Semantic Compatibility

↓

Highest Score

↓

Selected Antecedent

For example, both **Meena** and **Kavya** can match **“she”**, so semantic compatibility and discourse context are needed to prefer **Kavya**. This follows the assessment's required constraints: gender, number, recency, and semantic compatibility.

Q2 — Entity Chains and Discourse Coherence

Given text:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

Pseudocode

ALGORITHM CoherenceAnalysis(text)

INPUT: Discourse text

OUTPUT: Entity chains and coherence result

1. Divide text into sentences.

2. Extract entities and referring expressions

from each sentence.

3. FOR each expression R:
    Find possible antecedent entities.

    IF R matches an existing entity by
      semantic similarity and context:
        Link R to that entity.
    ELSE:
        Create a new entity.

4. Construct entity chains.

5. Calculate continuity:
    continuity ← number of linked references
            / total referring expressions

6. IF continuity is high:
    coherence ← "Coherent"
  ELSE:
    coherence ← "Low Coherence"

7. Return entity chains and coherence.
END ALGORITHM

Entity Chains

Chain 1 — Laptop

laptop
 ↓
The laptop
 ↓
it

↓

the computer

Chain 2 — Anjali

Anjali

↓

She

↓

Anjali

Other entities

processor

Python

machine learning application

Continuity

There are **5 main referring expressions**:

The laptop

She

it

Anjali

the computer

Most are linked to previously introduced entities, giving the discourse **high entity continuity**.

Therefore:

Coherence = HIGH

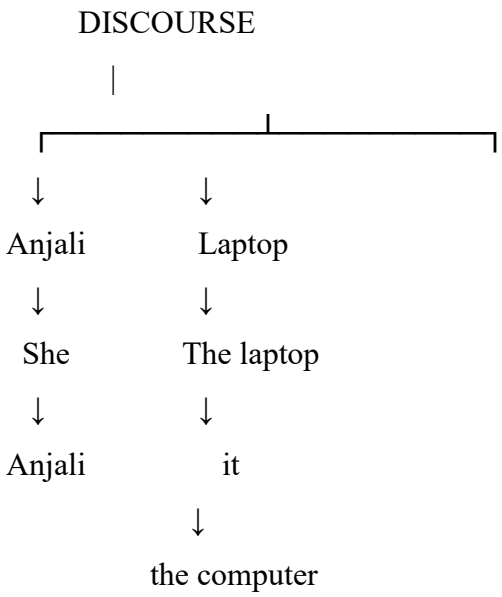
Why Breaking an Entity Chain Reduces Coherence

If **"it"** were incorrectly linked to **processor** instead of **laptop**, the sentence would become:

"She installed Python on the processor."

This creates an unlikely semantic relationship and breaks the **laptop → it → computer** entity chain. The discourse would therefore become less coherent.

Final Entity Structure



This follows the assessment's required steps: identify entities, link referring expressions, construct chains, evaluate continuity, and determine coherence.

Q3 — Algorithm for Identifying Discourse Relations

Given:

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Pseudocode

ALGORITHM DiscourseRelationAnalysis(text)

INPUT: Discourse text

OUTPUT: Discourse relations and structure

1. units ← Divide text into individual sentences.

2. FOR each consecutive pair (U1, U2):

    markers ← FindDiscourseMarkers(U2)

IF marker = "as a result":
relation ← Cause–Effect

ELSE IF marker = "after that":
relation ← Sequence

ELSE:
relation ← AnalyzeContext(U1, U2)

3. Construct discourse structure using
the identified relations.

4. Return discourse structure.

END ALGORITHM

Apply to the Given Text

Discourse Units

U1: The server crashed unexpectedly.

U2: As a result, users could not access the application.

U3: The technical team restarted the server.

U4: After that, the system became available again.

Relations

Units		**Marker**	**Relation**	**Reason**
U1 U2	→	**As a result**	**Cause–Effect**	Server crash caused users to lose access
U2 U3	→	—	**Problem–Solution**	Access problem is followed by the team's corrective action

Units		Marker	Relation	Reason
U3 U4	→	**After that**	**Sequence**	Availability followed the server restart

Discourse Structure

U1: Server crashed

↓ Cause

U2: Users could not access application

↓ Problem

U3: Team restarted server

↓ Sequence / Solution

U4: System became available

Why the Discourse Is Coherent

The sentences form a logical progression:

Problem

↓

Effect

↓

Solution

↓

Result

The explicit markers **"As a result"** and **"After that"** make the relationships clear, while the shared entity **server/system/application** maintains topic continuity. Therefore, the discourse has strong logical coherence.

Q4 — Algorithm for Dialogue Act Classification

Given conversation:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Pseudocode

```
ALGORITHM DialogueActClassification(conversation)

INPUT: Conversation utterances
OUTPUT: Dialogue-act sequence

1. FOR each utterance U in conversation:

    U ← Preprocess(U)

    features ← ExtractFeatures(U)
    // keywords, question marks, request words,
    // greeting words, offer expressions

    IF U contains greeting expression:
       act ← Greeting

    ELSE IF U is a question:
       act ← Question

    ELSE IF U expresses a request for help/action:
       act ← Request

    ELSE IF U offers help:
       act ← Offer
```

ELSE IF U provides information:

act ← Inform

ELSE IF U confirms something:

act ← Confirmation

Store act

2. RETURN dialogue-act sequence

END ALGORITHM

Apply to the Conversation

Utterance	Dialogue Act	Reason
"Hello, I need help with my assignment."	**Greeting + Request**	"Hello" is a greeting and "need help" is a request
"Sure, what problem are you facing?"	**Question**	Requests information from the user
"I do not understand machine learning."	**Inform**	Provides information about the user's problem
"I can explain the basic concepts to you."	**Offer**	Offers assistance

Dialogue-Act Sequence

User → Greeting + Request

Agent → Question

User → Inform

Agent → Offer

Why Dialogue-Act Recognition Helps

The conversational agent can use the recognized act to select an appropriate next response:

Request

↓

Ask for details

Inform

↓

Understand the problem

Offer

↓

Provide assistance

Thus, dialogue-act recognition helps the agent understand the **communicative intention** of each utterance and maintain an appropriate conversational flow.

Q5 — Algorithm for Dialogue State Tracking

Given conversation:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Pseudocode

ALGORITHM DialogueStateTracking(conversation)

INPUT: User-Agent conversation

OUTPUT: Updated dialogue state and next question

1. Initialize dialogue state:

Departure City ← NULL

Destination City ← NULL

Travel Date ← NULL

Number of Passengers ← NULL

2. FOR each user utterance U:

information ← ExtractFlightInformation(U)

IF departure city is found:
Departure City ← extracted city

IF destination city is found:
Destination City ← extracted city

IF travel date is found:
Travel Date ← extracted date

IF passenger number is found:
Number of Passengers ← extracted number

3. Identify missing slots.

4. IF Destination City is NULL:
Ask "Where would you like to travel?"

ELSE IF Departure City is NULL:
Ask "From which city?"

ELSE IF Travel Date is NULL:
Ask "What is your travel date?"

ELSE IF Number of Passengers is NULL:
Ask "How many passengers?"

ELSE:

Confirm the flight details.

5. Return updated dialogue state.

END ALGORITHM

Apply to the Conversation

Initial State

Departure City = NULL

Destination City = NULL

Travel Date = NULL

Number of Passengers = NULL

After:

“I want to go to Delhi.”

Departure City = NULL

Destination City = Delhi

Travel Date = NULL

Number of Passengers = NULL

Missing information:

Departure City

Travel Date

Number of Passengers

The agent asks:

“From which city?”

After:

“From Chennai.”

Final state so far:

Departure City = Chennai

Destination City = Delhi

Travel Date = NULL

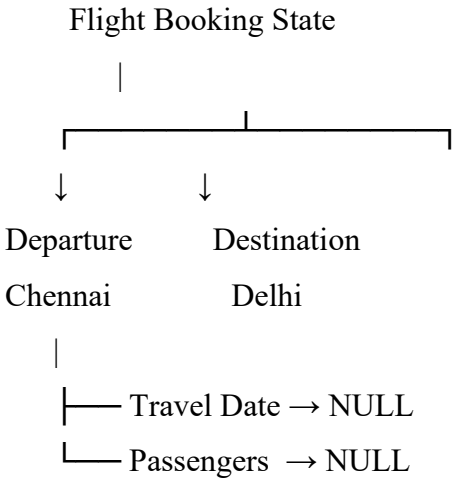
Number of Passengers = NULL

The next appropriate question is:

“What is your travel date?”

After that, the system should ask for the **number of passengers** if it has not been provided.

Final State Representation



Why Dialogue State Tracking Improves Coherence

The system **remembers information already provided** instead of asking the user to repeat it. Each new utterance updates the appropriate slot, while missing slots determine the next question. Thus, the conversation remains coherent across multiple turns and the agent can systematically collect all required booking information.